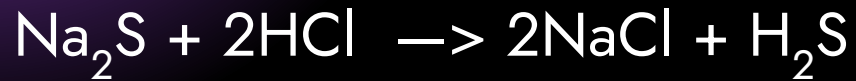


Double Displacement Reaction Solver

Adam Zinnekeh - Computer Engineering - May 6, 2026

What is a double displacement reaction?



Mass in = Mass Out
Due to Laws of Physics



$$\text{Fe} = 4$$

$$\text{O} = 6$$

$$\text{C} = 3$$

$$\text{Fe} = 4$$

$$\text{O} = 6$$

$$\text{C} = 3$$

Code Snippets:

```

struct Element {
    string symbol;    // (Ag) etc..
    double molarMass; // Mass in g/mol from the periodic table
    int ionCharge;    // The typical charge of the ion (e.g., +1, -2)
};

struct Compound {
    string formula;    // The final formatted string (e.g., "MgCl2")
    double totalMass;  // Sum of all atom masses in the balanced molecule
    int unitCharge;    // The magnitude of positive charge (used for balancing the equation)
};

```

```
//create our array for periodic table
Element pTable[50];
int totalElementsStored = 0;
//main function
```

```
void initializePTable() {
    pTable[0] = { "H", 1.008, 1 };
    pTable[3] = { "K", 39.098, 1 };
    pTable[6] = { "Mg", 24.305, 2 };
    pTable[9] = { "Fe", 55.845, 2 };
    pTable[12] = { "Pb", 207.2, 2 };
    pTable[15] = { "Cr3", 51.996, 3 };
    pTable[18] = { "Sr", 87.62, 2 };
    pTable[1] = { "Li", 6.941, 1 };
    pTable[4] = { "Ag", 107.868, 1 };
    pTable[7] = { "Ca", 40.078, 2 };
    pTable[10] = { "Cu", 63.546, 2 };
    pTable[13] = { "Al", 26.982, 3 };
    pTable[16] = { "Ni", 58.693, 2 };
    pTable[19] = { "Cs", 132.905, 1 };
    pTable[2] = { "Na", 22.990, 1 };
    pTable[5] = { "NH4", 18.044, 1 };
    pTable[8] = { "Ba", 137.327, 2 };
    pTable[11] = { "Zn", 65.38, 2 };
    pTable[14] = { "Fe3", 55.845, 3 };
    pTable[17] = { "Sn", 118.71, 2 };
    pTable[20] = { "F", 18.998, -1 };
    pTable[23] = { "I", 126.904, -1 };
    pTable[26] = { "ClO3", 83.45, -1 };
    pTable[29] = { "O", 15.999, -2 };
    pTable[32] = { "CO3", 60.008, -2 };
    pTable[35] = { "PO4", 94.971, -3 };
    pTable[21] = { "Cl", 35.450, -1 };
    pTable[24] = { "OH", 17.007, -1 };
    pTable[27] = { "C2H3O2", 59.04, -1 };
    pTable[30] = { "S", 32.060, -2 };
    pTable[33] = { "CrO4", 115.99, -2 };
    pTable[36] = { "C2O4", 88.02, -2 };
    pTable[22] = { "Br", 79.904, -1 };
    pTable[25] = { "NO3", 62.004, -1 };
    pTable[28] = { "MnO4", 94.94, -1 };
    pTable[31] = { "SO4", 96.060, -2 };
    pTable[34] = { "Cr2O7", 215.99, -2 };
    totalElementsStored = 37;
}
```

1 H Hydrogen																	2 He Helium														
3 Li Lithium	4 Be Beryllium																	10 Ne Neon													
11 Na Sodium	12 Mg Magnesium																	18 Ar Argon													
19 K Potassium	20 Ca Calcium	21 Sc Scandium																	36 Kr Krypton												
37 Rb Rubidium	38 Sr Strontium	39 Y Yttrium																	54 Xe Xenon												
55 Cs Caesium	56 Ba Barium	57 La Lanthanum																	86 Rn Radon												
87 Fr Francium	88 Ra Radium	89 Ac Actinium																	118 Og Oganesson												
																		5 B Boron	6 C Carbon	7 N Nitrogen	8 O Oxygen	9 F Fluorine	10 Ne Neon								
																		13 Al Aluminum	14 Si Silicon	15 P Phosphorus	16 S Sulfur	17 Cl Chlorine	18 Ar Argon								
																		29 Cu Copper	30 Zn Zinc	31 Ga Gallium	32 Ge Germanium	33 As Arsenic	34 Se Selenium	35 Br Bromine	36 Kr Krypton						
																		47 Ag Silver	48 Cd Cadmium	49 In Indium	50 Sn Tin	51 Sb Antimony	52 Te Tellurium	53 I Iodine	54 Xe Xenon						
																		79 Au Gold	80 Hg Mercury	81 Tl Thallium	82 Pb Lead	83 Bi Bismuth	84 Po Polonium	85 At Astatine	86 Rn Radon						
																		107 Rh Rhenium	108 Os Osmium	109 Ir Iridium	110 Pt Platinum	111 Au Gold	112 Hg Mercury	113 Tl Thallium	114 Pb Lead	115 Bi Bismuth	116 Po Polonium	117 At Astatine	118 Og Oganesson		
																		121 Ds Darmstadtium	122 Rg Roentgenium	123 Cn Copernicium	124 Nh Nihonium	125 Fl Flerovium	126 Mc Moscovium	127 Lv Livermorium	128 Ts Tennessine	129 Og Oganesson					
																		58 Ce Cerium	59 Pr Praseodymium	60 Nd Neodymium	61 Pm Promethium	62 Sm Samarium	63 Eu Europium	64 Gd Gadolinium	65 Tb Terbium	66 Dy Dysprosium	67 Ho Holmium	68 Er Erbium	69 Tm Thulium	70 Yb Ytterbium	71 Lu Lutetium
																		90 Th Thorium	91 Pa Protactinium	92 U Uranium	93 Np Neptunium	94 Pu Plutonium	95 Am Americium	96 Cm Curium	97 Bk Berkelium	98 Cf Californium	99 Es Einsteinium	100 Fm Fermium	101 Md Mendelevium	102 No Nobelium	103 Lr Lawrencium

- Alkali metals
- Alkaline earth metals
- Transition metals
- Post-transition metals
- Metalloids
- Reactive nonmetals
- Noble gases
- Lanthanides
- Actinides
- Unknown properties

Functions Needed...

```
// found a way to do greatest common divisor math
int getGCD(int a, int b) {
    a = abs(a);
    b = abs(b);
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

//least common multiple function
// where positive and negative charges cancel each other out globally.
int calculateLCM(int a, int b) {
    if (a == 0 || b == 0) return 0;
    return abs(a * b) / getGCD(a, b);
}
```

The Tricks we learned in Chemistry Class don't exactly work with Computer Logic...

Creating the Molecule from what we are Given

```
Compound makeMolecule(Element* cation, Element* anion) {  
    // the charges must equal each other on left and right, so we find the number of each ion needed by dividing their absolute value by our GCD to find the subscript  
    int divisor = getGCD(abs(cation->ionCharge), abs(anion->ionCharge));  
    int cSub = abs(anion->ionCharge) / divisor; // Number of cations needed  
    int aSub = abs(cation->ionCharge) / divisor; // Number of anions needed  
  
    // Formula string builder: combines the symbol and the subscript (if > 1).  
    string f = cation->symbol + (cSub > 1 ? to_string(cSub) : "");  
  
    //if our polyatomic atom needs to be multiplied (NO4) by two atoms, we just wrap that in parantheses  
    if (aSub > 1 && anion->symbol.length() > 2) {  
        f += "(" + anion->symbol + ")" + to_string(aSub);  
    }  
    else {  
        f += anion->symbol + (aSub > 1 ? to_string(aSub) : "");  
    }  
  
    // Molar Mass = mass1*#of1 + mass2 * # of 2  
    double mass = (cation->molarMass * cSub) + (anion->molarMass * aSub);  
  
    // unitCharge: The total internal charge magnitude of the unit. the absolute value of each charge * the number of cations needed  
    int chargeMag = abs(cation->ionCharge * cSub);  
  
    return { f, mass, chargeMag };  
}
```

```
Element* findInPTable(string userSymbol, int ionType) {  
    for (int i = 0; i < totalElementsStored; i++) {  
        if (pTable[i].symbol == userSymbol) {  
            // ionType 1 ensures it's a cation; ionType 2 ensures it's an anion.  
            if (ionType == 1 && pTable[i].ionCharge > 0) return &pTable[i];  
            if (ionType == 2 && pTable[i].ionCharge < 0) return &pTable[i];  
        }  
    }  
    return nullptr;  
}
```


How does it work?



```
initializePTable();

string c1, a1, c2, a2; //string that our chemical reaction is made from
cout << "Reactant 1 (Cation Anion): "; cin >> c1 >> a1;
cout << "Reactant 2 (Cation Anion): "; cin >> c2 >> a2;

//pointer to our ptable array search function, find each individual element
Element* eCation1 = findInPTable(c1, 1);
Element* eAnion1 = findInPTable(a1, 2);
Element* eCation2 = findInPTable(c2, 1);
Element* eAnion2 = findInPTable(a2, 2);
```

Utilizing Pointer to our Periodic Table to locate the Ion Needed

Building the Solution...

```
//check if we have a valid input
if (!eCation1 || !eAnion1 || !eCation2 || !eAnion2) {
    cout << "Error: Invalid entry. Ensure Cations (+) precede Anions (-)." << endl;
    return 1;
}

//create the reactants, then swap them
Compound reactant1 = makeMolecule(eCation1, eAnion1);
Compound reactant2 = makeMolecule(eCation2, eAnion2);
Compound product1 = makeMolecule(eCation1, eAnion2); // Swapped Product 1
Compound product2 = makeMolecule(eCation2, eAnion1); // Swapped Product 2

//math for balancing, we need to find a reaction least common multiple for the math to work out
int reactionLCM = calculateLCM(calculateLCM(reactant1.unitCharge, reactant2.unitCharge),
    calculateLCM(product1.unitCharge, product2.unitCharge));

// The coefficient is: (Global Target Charge) / (Charge of One Molecule).
int coeffR1 = reactionLCM / reactant1.unitCharge;
int coeffR2 = reactionLCM / reactant2.unitCharge;
int coeffP1 = reactionLCM / product1.unitCharge;
int coeffP2 = reactionLCM / product2.unitCharge;

//divide the coefficient that we calculate by our final greatest common divisor, finding the appropriate lowest coefficient for balance
int finalGCD = getGCD(getGCD(coeffR1, coeffR2), getGCD(coeffP1, coeffP2));
coeffR1 /= finalGCD; coeffR2 /= finalGCD;
coeffP1 /= finalGCD; coeffP2 /= finalGCD;
```



Masses Must Equal

Output.. Did we violate thermodynamics?

```
//output
cout << "\nBalanced Equation:\n";
// place each coefficient behind the string for the chemical formula as needed
cout << (coeffR1 > 1 ? to_string(coeffR1) : "") << reactant1.formula << " + "
    << (coeffR2 > 1 ? to_string(coeffR2) : "") << reactant2.formula << " -> "
    << (coeffP1 > 1 ? to_string(coeffP1) : "") << product1.formula << " + "
    << (coeffP2 > 1 ? to_string(coeffP2) : "") << product2.formula << endl;

// physics check:
cout << "\n--- Mass Balance Check ---" << endl;
cout << "Total Mass In:  " << (coeffR1 * reactant1.totalMass + coeffR2 * reactant2.totalMass) << " g/mol" << endl;
cout << "Total Mass Out: " << (coeffP1 * product1.totalMass + coeffP2 * product2.totalMass) << " g/mol" << endl;

return 0;
}

//func definitions
```

Reactant 1 (Cation Anion): Ag NO3

Reactant 2 (Cation Anion): Pb SO4

Balanced Equation:

2AgNO3 + PbSO4 -> Ag2SO4 + Pb(NO3)2

--- Mass Balance Check ---

Total Mass In: 643.004 g/mol

Total Mass Out: 643.004 g/mol